

Parallel Workflows in LangGraph

How to fan work out across multiple nodes that run at the same graph step, merge their results safely, and avoid the most common parallel-state pitfall in LangGraph.

WHAT THIS CHAPTER COVERS

- ▶ Fan-out / fan-in graph topology
- ▶ Non-LLM parallel workflow: Cricket Player Analysis
- ▶ The `InvalidUpdateError` and why it happens
- ▶ Partial-state updates vs. returning full state
- ▶ Reducer functions with `Annotated + operator.add`
- ▶ LLM-based parallel workflow: UPSC Essay Evaluator
- ▶ Structured output inside a parallel branch
- ▶ Sequential vs. Parallel workflow design

Parallel Workflows in LangGraph

RECAP — CHAPTER 5: SEQUENTIAL WORKFLOWS

Chapter 5 introduced the five-step LangGraph build pattern and showed how to chain nodes one after another using a linear sequence of edges — the BMI Calculator, a single-node LLM QA workflow, and a two-node Prompt Chaining blog generator. In every one of those graphs, exactly one node executed at a time and each node safely returned the entire updated state. This chapter removes that constraint: multiple nodes now execute at the *same* graph step, which changes how state must be written.

1. What Is a Parallel Workflow?

Definition

A *parallel workflow* is a LangGraph graph in which two or more nodes are connected to the same upstream node (often `START`) so that they all become eligible to execute at the same superstep, rather than waiting on one another. Their outputs are then combined by a downstream *join* or *summary* node before the graph continues.

Why It Exists

Many real computations are not inherently ordered. If three quantities can each be derived directly from the same input data — without needing each other's results first — forcing them into a sequential chain wastes time and adds artificial dependencies. LangGraph lets independent branches of a graph run concurrently because a graph, unlike a strict pipeline, only encodes the dependencies that actually exist.

Problem It Solves

Sequential chaining (Chapter 5) assumes every step needs the output of the step before it. Parallel workflows remove that assumption for genuinely independent computations, and they also enable a very common real-world pattern: sending the same input to multiple LLM calls that each evaluate a different aspect of it, then merging the individual verdicts into one final judgment.

How It Works

1. One or more nodes are connected directly from `START` (or from a common ancestor) using separate `add_edge()` calls — this is the **fan-out**.
2. LangGraph's execution engine (Pregel-inspired, see Chapter 4) runs every node with no unresolved dependency in the same superstep.
3. All parallel branches connect to the same downstream node — this is the **fan-in** or join, where their combined outputs become visible together for the first time.

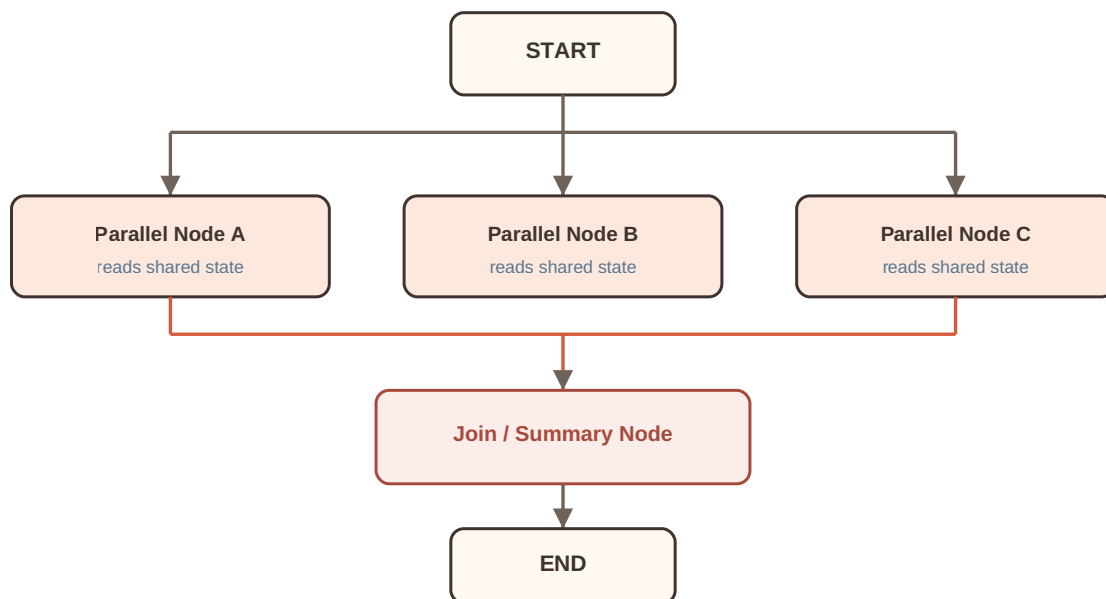


Figure 6.1 — Generic fan-out / fan-in topology: independent nodes execute in the same superstep before a join node merges their results.

How This Differs From Chapter 5

Aspect	Sequential Workflow (Ch. 5)	Parallel Workflow (Ch. 6)
Node dependency	Each node depends on the previous node's output	Sibling nodes are mutually independent
Execution order	Strictly one node per superstep	Multiple nodes in the same superstep
Typical edge pattern	$A \rightarrow B \rightarrow C$	$START \rightarrow \{A, B, C\} \rightarrow \text{join}$
State return	Safe to return the entire state dict	Must return only the keys the node actually computed

🔄 Quick Revision — Parallel Workflows

- A parallel workflow connects multiple nodes to the same upstream node so they run in the same superstep.
- Parallel branches must be genuinely independent — none should need another branch's output.
- Fan-out = one node feeding multiple next nodes; fan-in = multiple nodes feeding one join node.
- LangGraph's Pregel-inspired execution model is what allows nodes with no unresolved dependency to run together.

2. Building a Non-LLM Parallel Workflow: Cricket Player Analysis

Problem Statement

Given a batsman's raw innings data — runs, balls faced, fours, and sixes — compute three independent derived statistics: **strike rate**, **boundary percentage** (what share of total runs came from boundaries), and **balls per boundary** (how often, on average, the batsman hits a boundary). None of these three quantities depends on either of the other two — each is calculated directly from the same four raw inputs — which makes this a natural fit for a parallel workflow.

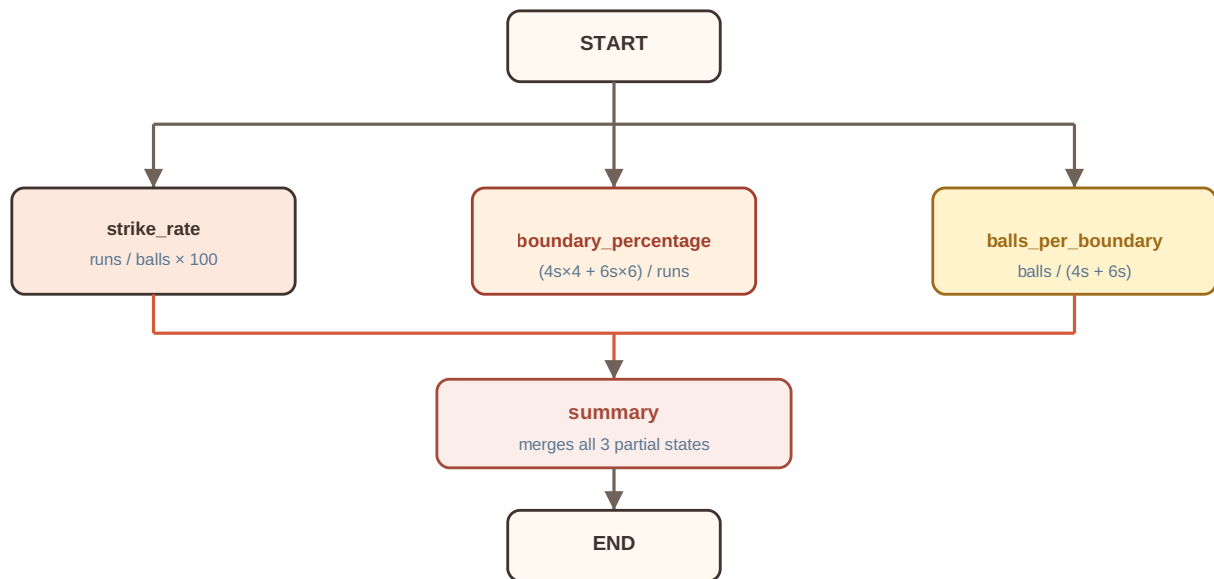


Figure 6.2 — Cricket Player Analysis workflow: three independent stat-calculation nodes fan out from *START* and fan into a single *summary* node.

State Definition

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict

class PlayerState(TypedDict):
    runs : int
    balls : int
    no_of_4 : int
    no_of_6 : int

    strike_rate : float
    boundary_percentage : float
    balls_per_boundary : float
    summary : str
```

Node Functions

Each calculation node reads the raw inputs it needs from `state` and returns **only the key it computed**, as a small dictionary — not the full `state` object.

```
def calculate_strike_rate(state : PlayerState):
    runs = state["runs"]
    balls = state["balls"]

    strike_rate = round((runs/balls)*100,2)

    return {"strike_rate":strike_rate}

def calculate_boundary_percentage(state : PlayerState):
    runs_from_fours = state["no_of_4"]*4
    runs_from_sixes = state["no_of_6"]*6
    total_runs = state["runs"]

    boundary_percentage = round(((runs_from_fours + runs_from_sixes)/total_runs) * 100, 2)

    return {"boundary_percentage":boundary_percentage}

def calculate_balls_per_boundary(state : PlayerState):
    balls_for_4 = state["no_of_4"]
    balls_for_6 = state["no_of_6"]
    total_balls = state["balls"]

    balls_per_boundary = round(total_balls/(balls_for_4 + balls_for_6), 2)

    return {"balls_per_boundary":balls_per_boundary}

def generate_summary(state : PlayerState):

    summary = f"""
    Strike Rate : {state["strike_rate"]} \n
    Boundary Percentage : {state['boundary_percentage']} \n
    Balls per Boundary : {state["balls_per_boundary"]}"""

    return {"summary":summary}
```

Graph Assembly — Nodes and Edges

The graph has four nodes. The fan-out is three separate `add_edge(START, ...)` calls; the fan-in is three separate edges into `"summary"`.

```

graph = StateGraph(PlayerState)

# Nodes
graph.add_node("strike_rate", calculate_strike_rate)
graph.add_node("boundary_percentage", calculate_boundary_percentage)
graph.add_node("balls_per_boundary", calculate_balls_per_boundary)
graph.add_node("summary", generate_summary)

# Edges
graph.add_edge(START, "strike_rate")
graph.add_edge(START, "boundary_percentage")
graph.add_edge(START, "balls_per_boundary")

graph.add_edge("strike_rate", "summary")
graph.add_edge("boundary_percentage", "summary")
graph.add_edge("balls_per_boundary", "summary")

graph.add_edge("summary", END)

workflow = graph.compile()

```

Execution

```

initial_state = {
    "runs":75,
    "balls":33,
    "no_of_4":4,
    "no_of_6":7
}

workflow.invoke(initial_state)

# {'runs': 75, 'balls': 33, 'no_of_4': 4, 'no_of_6': 7,
#  'strike_rate': 227.27, 'boundary_percentage': 77.33,
#  'balls_per_boundary': 3.0,
#  'summary': '\n    Strike Rate : 227.27 \n\n    Boundary Percentage : 77.33 \n\n    Balls
per Boundary : 3.0'}

```

INTERVIEW TIP

If asked "how does LangGraph know these three nodes can run in parallel?", the correct answer is structural, not magical: it is entirely determined by the edges you draw. Three nodes fed directly from **START** with no edges between them have no declared dependency on each other, so the engine is free to execute them in the same superstep.

🔄 Quick Revision — Cricket Player Analysis Workflow

- Three independent stat nodes (strike rate, boundary %, balls per boundary) fan out from START.
- All four raw inputs (runs, balls, fours, sixes) are read by more than one parallel node without conflict — reads never conflict, only writes do.
- Each node returns a partial-state dict containing only the key it computed.
- A single summary node fans back in from all three branches and formats the final report.

3. The InvalidUpdateError: Full State vs. Partial State

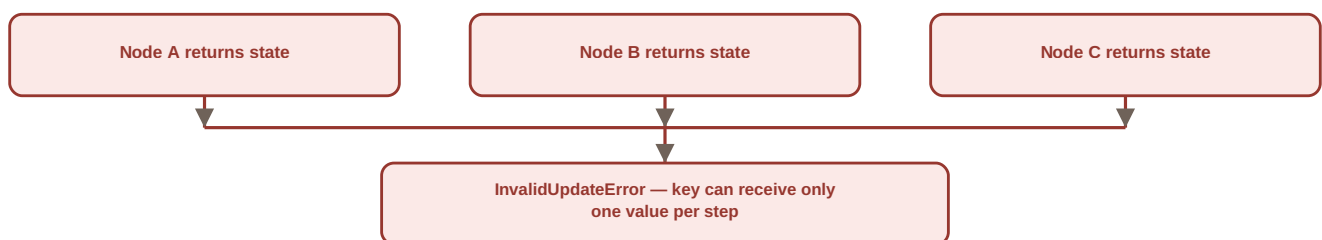
The Problem

In every sequential workflow so far, a node was free to return the entire `state` dictionary because only one node ever wrote to state at a given step. In a parallel workflow this breaks: if two or more nodes belonging to the same superstep each return the *full* state object, LangGraph sees multiple simultaneous writes to every key in that state — including keys the node never actually changed — and raises:

```
langgraph.errors.InvalidUpdateError: At key 'key_name':  
Can receive only one value per step
```

Even though a node like `calculate_strike_rate` only ever reads `runs` and `balls` and never modifies them, returning the whole `state` object makes LangGraph treat every key in it as if it had been written by that node. With three parallel nodes all doing this for the same shared keys, the graph cannot determine which of the three conflicting "writes" is correct.

WRONG — EACH NODE RETURNS THE FULL STATE



CORRECT — EACH NODE RETURNS A PARTIAL-STATE DICT

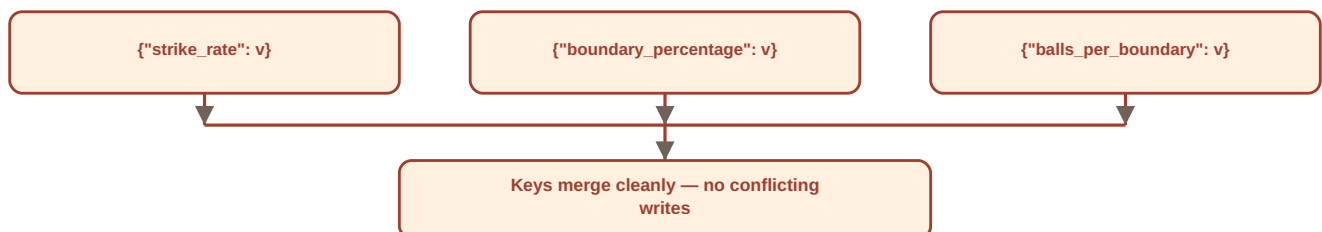


Figure 6.3 — Returning the full state from every parallel branch causes conflicting writes; returning only the changed keys resolves it cleanly.

The Fix — Partial-State Updates

Instead of returning `state`, each node returns a small dictionary containing only the key(s) it actually computed:

WRONG — CAUSES INVALIDUPDATEERROR

```
def calculate_strike_rate(state):
    state["strike_rate"] = round(
        (state["runs"]/state["balls"])*100, 2
    )
    # returns the ENTIRE state
    return state
```

CORRECT — PARTIAL UPDATE

```
def calculate_strike_rate(state):
    strike_rate = round(
        (state["runs"]/state["balls"])*100, 2
    )
    # returns only the new key
    return {"strike_rate": strike_rate}
```

IMPORTANT

Every LangGraph node function is a plain Python function that accepts a dictionary (the state) as input and can return any dictionary as output — it is not required to return the same `state` object it received. Returning a smaller, partial dictionary is completely valid and is the only safe pattern once more than one node can write in the same superstep.

COMMON MISTAKE

A very natural but incorrect assumption is that sequential-style state handling ("always return the full state") is a universal LangGraph rule. It only works when exactly one node executes per superstep. As soon as parallel branches are introduced, returning full state is a latent bug that may not surface until you add a second parallel node.

BEST PRACTICE

Adopt partial-state returns as the default habit for every node you write, sequential or parallel. It works correctly in both cases, it makes each node's side effects explicit and easy to audit, and it removes the need to remember which workflows require it.

🔄 Quick Revision — InvalidUpdateError

- `InvalidUpdateError` fires when two or more nodes in the same superstep both write to the same state key.
- Returning the full state object counts as writing every key in it, even unchanged ones.
- The fix is to return a dictionary containing only the keys a node actually computed — a partial-state update.
- Partial-state updates are safe in both sequential and parallel workflows; full-state returns are only safe when nodes never run concurrently.

4. Reducer Functions: Merging Parallel Writes Into One Key

Definition

A *reducer* is a function attached to a state key (via `Annotated[...]`) that tells LangGraph how to **combine** multiple simultaneous writes to that key, instead of one write simply overwriting another. Reducers were introduced conceptually in Chapter 4; this chapter is where they become necessary in practice.

Why It Exists

Partial-state updates solve the conflict problem for keys that only one branch writes to. But sometimes several parallel branches all need to contribute to the *same* key — for example, three LLM evaluator nodes each producing one numeric score that must all land inside a single shared list. Without a reducer, the default behaviour for a repeated key is replace, so only the last write would survive and the other two scores would be silently lost.

How It Works

The key is typed with `Annotated[list[int], operator.add]`. `operator.add` is the functional form of the `+` operator, imported from Python's built-in `operator` module. When two or more parallel nodes each return `{"individual_score": [n]}`, LangGraph does not overwrite the list — it calls the reducer to merge the writes, effectively performing `[8] + [7] + [6]` to produce `[8, 7, 6]`.

```
import operator
from typing import TypedDict, Annotated

class FeedbackState(TypedDict):
    essay : str
    CoT_feedback : str
    DoA_feedback : str
    language_feedback : str
    final_feedback : str
    individual_score : Annotated[list[int], operator.add]
    final_score : float
```

```
individual_score : Annotated[list[int], operator.add]
```

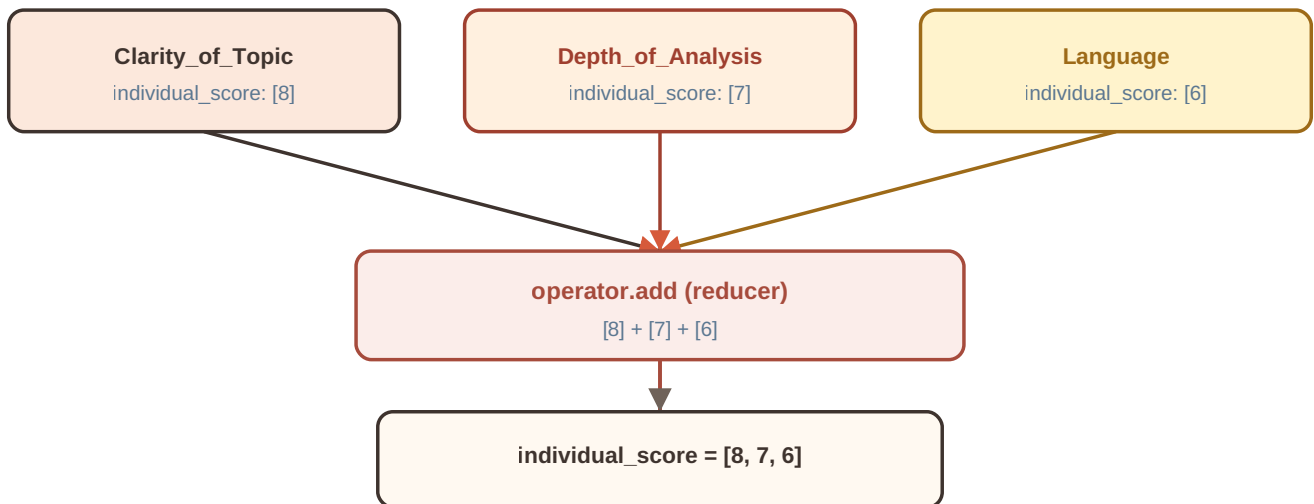


Figure 6.4 — Three parallel evaluator nodes each write a single-item list; the `operator.add` reducer merges them into one combined list.

REAL WORLD INSIGHT

LangGraph ships several reducer-compatible functions beyond `operator.add` (for lists, counters, or custom merge logic), and you can write your own reducer function when the merge logic is more involved than concatenation — for example, keeping only the maximum score, or deduplicating merged results. The lecture defers a deeper look at custom reducers to a later chapter, but the underlying idea is always the same: a reducer decides how N simultaneous writes to one key become one final value.

🔄 Quick Revision — Reducer Functions

- A reducer tells LangGraph how to merge multiple simultaneous writes to the same state key.
- Without a reducer, a repeated key simply gets overwritten — only the last write survives.
- `Annotated[list[int], operator.add]` merges parallel single-item list writes via list concatenation.
- `operator.add` is the functional form of `+` from Python's built-in operator module.
- Reducers matter only for keys that multiple parallel branches write to — unique per-branch keys need no reducer.

5. Building an LLM-Based Parallel Workflow: UPSC Essay Evaluator

Problem Statement

Given a UPSC-style essay, evaluate it along three independent dimensions — **Clarity of Thought**, **Depth of Analysis**, and **Language** — using a separate LLM call per dimension. Each call must return both a textual feedback and a numeric score out of 10. A final node merges the three text feedbacks into one summarized verdict and averages the three scores into a final score.

This workflow combines three ideas from earlier in the course: **parallel fan-out/fan-in** (this chapter), **structured output via Pydantic schemas** (LangChain playlist), and **reducer functions** (Section 4 above).

Structured Output Schema

Because three separate LLM calls each need to reliably return a feedback string plus a numeric score, free-form prompting is not safe enough — the model could occasionally return a score as text ("seven") instead of a number. A Pydantic schema, enforced via a structured output parser, guarantees a consistent shape every time.

```
from pydantic import BaseModel, Field
from langchain_core.output_parsers import PydanticOutputParser, StrOutputParser

# Output Parsers
class LLMOutput(BaseModel):
    feedback : str = Field(description = "Feedback of the essay based on the given metric.")
    rating : str = Field(ge = 0, le = 10, description = "Rating of the essay based on the given metric")

parser1 = PydanticOutputParser(pydantic_object = LLMOutput)
parser2 = StrOutputParser()
```

Prompts

```
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate(
    template = """
    You are a professional UPSC interviewer who reviews the responses of aspirants and scores them.
    Carefully provide a feedback and rate out of 10 the following essay :- \n\n {essay} \n\n
    Based on the {metric} metric.""",
    input_variables = ["essay", "metric"],
    partial_variables = {"format_instructions": parser1.get_format_instructions()}
)

evaluate_prompt = PromptTemplate(
    template = """Give a final text feedback for the given essay : \n {essay} \n
    based on the different types of feedbacks : \n
    Clarity of topic - {CoT}
    Depth of analysis - {DoA}
    Language - {lang}""",
    input_variables = ["essay", "CoT", "DoA", "lang"]
)
```

State Definition

```
class FeedbackState(TypedDict):
    essay : str
    CoT_feedback : str
    DoA_feedback : str
    language_feedback : str
    final_feedback : str
    individual_score : Annotated[list[int], operator.add]
    final_score : float
```

IMPORTANT

Notice that `CoT_feedback`, `DoA_feedback`, and `language_feedback` are three *separate* keys, each written by exactly one node — so none of them need a reducer. Only `individual_score` is written by all three parallel nodes into the same key, which is precisely why it alone needs `Annotated[ ..., operator.add]`.

🔄 Quick Revision — UPSC Workflow Setup

- The workflow evaluates one essay across three independent LLM-scored dimensions: Clarity, Depth, Language.
- A Pydantic schema (`LLMOutput`) plus `PydanticOutputParser` guarantees consistent `{feedback, rating}` structure per call.
- `FeedbackState` holds three separate per-dimension feedback keys (no reducer needed) plus one shared `individual_score` list (reducer required).
- This workflow layers three concepts together: parallel fan-out/fan-in, structured output, and reducer functions.

6. Assembling the Full UPSC Workflow

Node Functions

Each evaluator node builds a small chain — `prompt | llm | parser1` — invokes it with the essay and the relevant metric, and returns a partial-state update containing that dimension's feedback plus a single-item score list (merged later by the reducer).

```

def rate_on_CoT(state : FeedbackState):
    essay = state["essay"]
    chain = prompt | llm | parser1

    response = chain.invoke({
        "essay": essay,
        "metric": "Clarity of the Topic"
    })

    return {"CoT_feedback": response.feedback, "individual_score": [int(response.rating)]}

def rate_on_DoA(state : FeedbackState):
    essay = state["essay"]
    chain = prompt | llm | parser1
    response = chain.invoke({
        "essay": essay,
        "metric": "Depth of Topic"
    })

    return {"DoA_feedback": response.feedback, "individual_score": [int(response.rating)]}

def rate_on_language(state : FeedbackState):
    essay = state["essay"]
    chain = prompt | llm | parser1
    response = chain.invoke({
        "essay": essay,
        "metric": "Language"
    })

    return {"language_feedback": response.feedback, "individual_score":
[int(response.rating)]}

```

The Join Node: Final Evaluation

The join node runs only after all three evaluator branches have written their outputs. It uses the *unstructured* model (parser2 / StrOutputParser) rather than the structured one, since the summarized feedback is free-form text, not another {feedback, score} record. The final score is a plain Python average of the merged `individual_score` list.

```
def final_evaluation(state : FeedbackState):
    essay = state["essay"]
    CoT = state["CoT_feedback"]
    DoA = state["DoA_feedback"]
    lang = state["language_feedback"]

    # Summarized Feedback (plain string output, not structured)
    chain = evaluate_prompt | llm | parser2
    response = chain.invoke({"essay":essay, "CoT":CoT, "DoA":DoA, "lang":lang})

    # Average Score
    final_score = sum(state["individual_score"])/len(state["individual_score"])

    return {"final_feedback": response, "final_score": final_score}
```

Graph Assembly

```
# Nodes
graph = StateGraph(FeedbackState)

graph.add_node("Clarity_of_Topic", rate_on_CoT)
graph.add_node("Depth_of_Analysis", rate_on_DoA)
graph.add_node("Language", rate_on_language)
graph.add_node("Final_Evaluation", final_evaluation)

# Edges
graph.add_edge(START, "Clarity_of_Topic")
graph.add_edge(START, "Depth_of_Analysis")
graph.add_edge(START, "Language")

graph.add_edge("Clarity_of_Topic", "Final_Evaluation")
graph.add_edge("Depth_of_Analysis", "Final_Evaluation")
graph.add_edge("Language", "Final_Evaluation")
graph.add_edge("Final_Evaluation", END)

workflow = graph.compile()
```

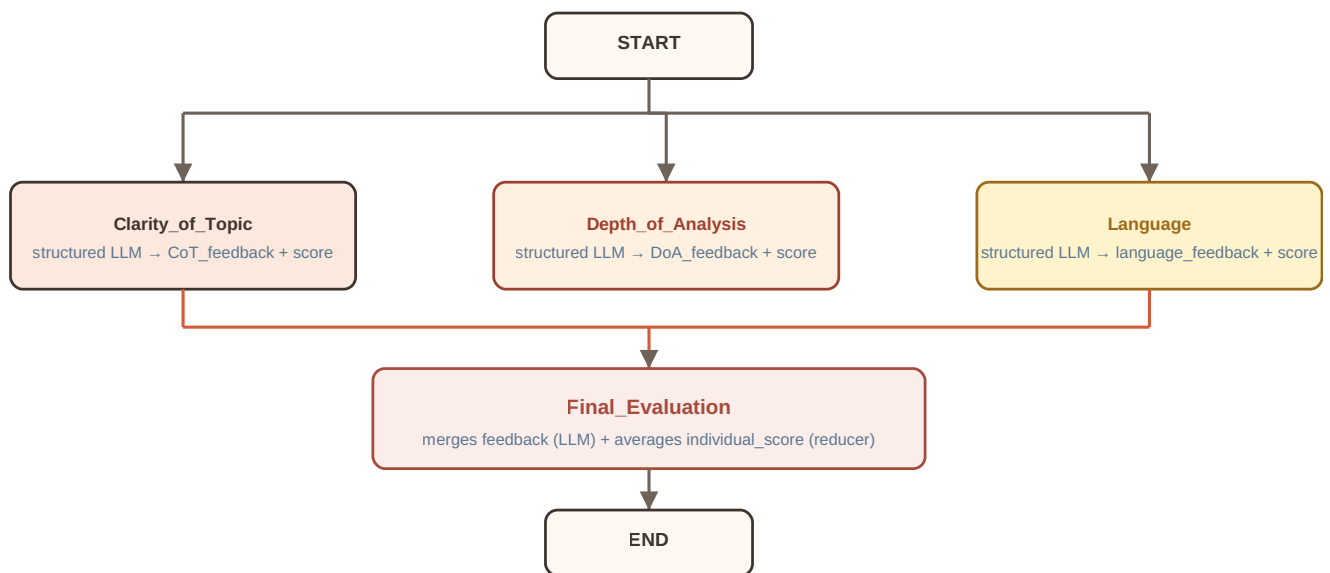


Figure 6.5 — UPSC Essay Evaluator: three structured-output LLM nodes evaluate the essay in parallel; *Final_Evaluation* merges feedback and averages scores.

Execution

```

initial_state = {"essay":essay}

workflow.invoke(initial_state)

# individual_score comes back merged as a list, e.g. [7, 8, 8]
# final_score is the plain average of that list
# final_feedback is the LLM-generated summarized verdict

```

INTERVIEW TIP

A good way to demonstrate understanding of this workflow in an interview is to explain *why* the join node cannot start early: LangGraph will not execute `Final_Evaluation` until all three of its declared incoming edges (`Clarity_of_Topic`, `Depth_of_Analysis`, `Language`) have completed, so the merge is guaranteed to see all three scores and all three feedback strings before it runs.

🔄 Quick Revision — Full UPSC Workflow

- Three evaluator nodes each run `prompt | llm | parser1 (structured output)` for one metric and return a partial update.
- The join node (`Final_Evaluation`) only runs after all three incoming edges have completed.
- `Final_Evaluation` uses the plain `StrOutputParser` chain to summarize feedback, and averages the reducer-merged `individual_score` list.
- The same essay, sent to three prompts differing only by the `{metric}` variable, is a lightweight way to reuse one prompt template across parallel branches.

7. Sequential vs. Parallel: Choosing the Right Topology

Comparison

Aspect	Sequential Workflow	Parallel Workflow
When to use	Step B genuinely needs step A's output	Steps are independently derivable from the same input
State return style	Full state is safe (only one writer per step)	Partial state is required (multiple writers per step)
Shared-key writes	Not a concern — one node at a time	Need a reducer if more than one branch writes the same key
Latency	Sum of every node's latency	Roughly the slowest single branch's latency
Typical use case	Translate → summarize → format	Multi-metric scoring, multi-perspective evaluation

BEST PRACTICE

Before wiring a graph, explicitly ask of every pair of candidate nodes: "does either one need the other's output?" If the answer is no for a whole group of nodes, connect them as parallel branches from a common ancestor rather than defaulting to a sequential chain out of habit — the parallel version is both faster and a more honest model of the actual dependency structure.

🔄 Quick Revision — Sequential vs. Parallel

- Choose sequential when a later step genuinely needs an earlier step's output; choose parallel when steps are independent.
- Parallel workflows can reduce end-to-end latency to roughly the slowest branch instead of the sum of every step.
- Partial-state returns and reducers are the two mechanical requirements that make parallel workflows safe.
- The graph topology should reflect the real dependency structure of the computation, not an assumed default.

K. Key Takeaways

- **Parallel workflows** connect multiple mutually-independent nodes to a common ancestor so they execute in the same LangGraph superstep, then merge back into a join node.
- **Partial-state updates** — returning only the keys a node actually computed, instead of the full state dict — are mandatory once more than one node can write in the same step.
- **InvalidUpdateError** occurs when two or more parallel nodes both write (or appear to write, via full-state returns) to the same key in one superstep.
- **Reducer functions**, attached via `Annotated[type, reducer_fn]`, tell LangGraph how to merge multiple simultaneous writes to one shared key — e.g. `operator.add` concatenates parallel single-item lists.
- The **Cricket Player Analysis workflow** is a non-LLM example: three independent stat calculations fan out from `START` and fan into a summary node.
- The **UPSC Essay Evaluator workflow** is an LLM-based example combining parallel fan-out/fan-in, Pydantic structured output, and a reducer for merging three evaluator scores.
- Choosing between sequential and parallel topology should be driven by the real dependency structure of the computation, not by habit.

R. Revision Sheet

Parallel Workflow

A graph where multiple nodes share a common ancestor and execute in the same superstep, later merging at a join node.

InvalidUpdateError

Raised when a state key receives more than one conflicting write within a single superstep.

Fan-out / Fan-in

Fan-out: one node feeding several next nodes. Fan-in: several nodes feeding one downstream node.

Reducer Function

A merge function attached to a state key via `Annotated[type, fn]` that combines multiple simultaneous writes instead of overwriting.

Partial-State Update

A node returns a dict containing only the keys it computed, instead of the entire state object.

`operator.add`

The functional form of `+`; as a reducer it concatenates parallel single-item lists into one combined list.

Cricket Player Analysis

Non-LLM example: strike rate, boundary %, and balls-per-boundary computed in parallel from shared raw stats.

UPSC Essay Evaluator

LLM-based example: three structured-output evaluator nodes score an essay in parallel; a join node merges feedback and averages scores.

I. Interview Questions

Q1. What structurally makes a set of LangGraph nodes eligible to run in the same superstep?

Q2. Why does returning the full state from a node become unsafe once parallel branches are introduced?

Q3. Explain the `InvalidUpdateError`: what causes it, and how do you fix it?

Q4. What is a reducer function, and when does a state key actually need one?

Q5. Walk through what `Annotated[list[int], operator.add]` does when three parallel nodes each write a single-item list to that key.

Q6. In the Cricket Player Analysis workflow, why don't the three stat-calculation nodes need a reducer even though they all read from the same input keys?

Q7. Why does the UPSC Essay Evaluator workflow use a structured-output model for the three evaluator nodes but a plain string-output model for the final join node?

Q8. What guarantees that the `Final_Evaluation` node in the UPSC workflow only runs after all three evaluator nodes have completed?

Q9. How would you decide whether two computational steps belong in a sequential chain or in parallel branches?

Q10. What latency advantage does a parallel workflow offer over an equivalent sequential workflow, and why?

F. Further Reading

- LangGraph official documentation — Graph API: nodes, edges, and state (langchain-ai.github.io/langgraph)
- LangGraph documentation — state reducers and Annotated types
- Python standard library documentation — the `operator` module
- LangChain documentation — structured output with Pydantic and output parsers
- Google Research, "The Pregel Model" — background on the superstep-based execution model LangGraph draws on